Day 49 – OData Adapter: Filter, Expand, Batch Processing & V2 vs V4

Class date: 6th September 2026 | Module: Adapters (OData)

Correction to your V2 vs. V4 comparison table: you wrote that **OData V2 "does not support JSON format"** (Atom/XML only). This isn't accurate – V2 genuinely does support JSON as a response format (via **$format=json**), it simply **defaults** to Atom/XML unless JSON is explicitly requested. The real, precise differences for your table are: V2's *default* is Atom/XML while V4's default (and only supported) format is JSON; and the other three rows in your table (Root Entity vs. Expand Entities, Patch support, Performance) are correct, with a bit more precision added in Section 6 below.

1. Recap (start here first)

The 8 standard OData query options were introduced on Day 48; two were left pending – **$filter** and **$expand**. Both are covered fully today.

2. $filter – filtering data

$filter lets you filter out and retrieve only **some** of the data, based on a condition you specify – i.e. it tells the service *on what basis* to filter.

Example used in class – the Order Information Entity, filtering on **ShipCountry = 'France'**:

```
?$filter=ShipCountry eq France
```

eq is the comparison operator used here, meaning "equal to" – this is the operator covered in class today. (Other standard OData comparison operators exist too, such as **ne** (not equal), **gt/lt** (greater/less than), for future reference.)

Result: only records where ShipCountry = France are returned in the response.

Why this matters – performance: instead of fetching the complete payload and then filtering it afterward inside the iFlow, using **$filter** means only the required records come back from the source system directly. Example illustrated in class: out of **100 records**, if only **1 record** is actually needed, filtering at the source means a **faster response time** (less data to transfer and process) compared to pulling all 100 and discarding 99 afterward. Similarly: out of **200 records**, filtering down to **10 records** directly at the source is far more efficient.

$filter also supports text/substring-based conditions – e.g. filtering where a text field contains, starts with, or ends with a given substring (covered at a high level; specific substring function syntax to be explored hands-on in a future class).

Bottom line: instead of getting the complete payload and filtering it yourself afterward, use the **Filter query** to get only the required records directly.

3. $expand – pulling in related data

Expand, basically, means to **enlarge** – i.e. get **more than** the data an Entity would normally return on its own.

Example used in class: calling Employee details normally does **not** bring back Address information – because **Employee API**, **Address API**, and **Projects API** are all **separate APIs/Entities**. But suppose

the requirement is to get **Address** and **Projects** details together **with a single Employee API call**, rather than calling three separate APIs.

Solution: Navigation Property (important term). An Entity can define Navigation Properties that point to related **sub-entities**:

```
Employee (Entity) |-- Address (sub-entity, via Navigation Property) |-- Projects (sub-entity, via Navigation Property)
```

In other words: **Root Entity** → **Sub-Entity**, connected via the **Expand operation** in OData.

Second example used in class: Customers → Orders (enlarging the Customer entity to include its related Orders):

```
Customers?$expand=Orders (E1 = Customers, the root Entity; E2 = Orders, the related sub-entity being pulled in)
```

Key point: if you want sub-entity details, you use the **Expand operation** – and critically, this brings back everything in a **single call**, rather than needing multiple separate API calls.

4. Demo – calling OData from SAP CPI (Receiver Adapter)

Connection tab fields used in the demo:

- **Address** – the service endpoint.
- **Resource Path** – the specific Entity/resource being called.
- **EDMX file** (the metadata file) – can be provided as either a **Remote** reference (CPI fetches it live from the service) or a **Local** EDMX file (uploaded manually, same idea as uploading a WSDL for SOAP).

"Connect to System" step: provide the **Address** as **Service Root URL + /$metadata**, exactly as covered on Day 48, so CPI can pull the EDMX schema directly.

Step-by-step flow followed in the demo:

1. **Step 1:** Connect to the system using the Service Root URL + $metadata.
2. **Step 2:** Choose **how many Entities** are needed for this call, and select the required fields from each.
3. **CPI automatically generates the query parameters** (e.g. the $select list) based on exactly which fields you tick/select in the adapter UI – you don't have to hand-write the query string yourself.
4. **Step 3: Filter By / Sort By** – also configured through the adapter UI; again, **SAP CPI generates the underlying query** (the $filter/$orderby syntax) automatically based on your selections. A related option covers **Sub-levels** → **Expand**, for pulling in related sub-entities the same way.

OData adapter behaviour: it automatically **removes unwanted headers and fields** from the response, returning only what was actually requested.

Other adapter-level notes from the demo:

- **Custom query options** can still be added manually alongside the standard, adapter-generated ones, if the target service supports application-specific parameters.
- **Atom** is one type of XML format (used by OData V2 responses by default, as covered on Day 48).
- **Process in Pages** (checkbox) – processes the result set page by page rather than all at once, working hand-in-hand with the $top/$skip pagination concept from Day 48.

- **Attach Error Details on Failure** – same concept as covered for the HTTP Receiver adapter (Day 42): attaches request/response details to help debugging when a call fails.

5. Calling multiple receivers together – Multicast pattern

Example scenario: Employee information needs to be sent to **two different receivers** – one expecting an **OData (GET)** call, and another expecting a plain **HTTPS (GET)** call.

Multicast is the step used for this: after a Sender/Timer starts the iFlow, a **Multicast** step splits the processing into **parallel Request-Reply branches** – one branch calling the OData receiver, the other calling the HTTPS receiver – both from the **same triggering event**.

```
Sender/Timer -> Multicast -> Branch 1: Request-Reply -> OData (GET) -> Receiver 1 ->
Branch 2: Request-Reply -> HTTPS (GET) -> Receiver 2
```

6. Batch Processing

Batch Processing is used when you want to perform **multiple operations in a single call**, instead of making one call per operation.

Example: Create, Update, and Delete operations on three different employee records (Emp1 → Create, Emp2 → Update, Emp3 → Delete) can all be sent together as **one batch call**, rather than three separate calls.

Important to remember: Batch processing applies to **write** operations (Create/Update/Delete). A plain **GET call (read)** does **not** use Batch Processing – it remains a normal, direct call.

Parts of a batch request:

- **Batch (the overall envelope)**, containing one or more **Change Sets**.
- **Change Set Part** – each individual operation within the batch, made up of a **Method** (Create/Update/Delete) and a **URI** (which record/entity it targets).

Why batch processing matters – cost/efficiency comparison from class:

Approach	Behaviour	Cost
Splitter approach	10 messages split out individually → 10 separate API calls	10× the API call cost
Batch Processing	1 single call carrying 10 operations	1× the API call cost

This is exactly why Batch Processing is preferred for bulk write operations in real projects – it directly reduces the number of billed/rate-limited API calls made against the target system.

7. OData V2 vs. V4 – key differences (corrected)

Aspect	OData V2	OData V4
Response format	Defaults to Atom/XML; JSON is also available via $format=json (not "unsupported," just not the default).	Defaults to — and only supports — JSON.

Query options scope	Standard query options ($filter, $select, $orderby, etc.) work only at the Root Entity level.	Query options can also be applied on Expanded (related) Entities, not just the root.
Partial update	No native PATCH verb; partial updates are done via POST with an X-HTTP-METHOD: MERGE header.	Native HTTP PATCH method is supported directly for partial updates.
Performance / capabilities	Comparatively less efficient; fewer built-in capabilities.	Generally more performant, plus extra features like delta queries (fetching only what changed since the last call).

Quick-recall summary

- **$filter** = get only matching records at the source (e.g. **?$filter=ShipCountry eq France**) – faster than fetching everything and filtering afterward.
- **$expand** = pull in related sub-entities (via **Navigation Properties**) in a single call, e.g. **?$expand=Orders**.
- In the CPI OData adapter, **CPI auto-generates the query string** ($select/$filter/$orderby/$expand) based on what you select in the UI – you rarely hand-write it.
- **Multicast** lets one trigger fan out to multiple receivers (e.g. OData GET + HTTPS GET) in parallel Request-Reply branches.
- **Batch Processing** bundles multiple Create/Update/Delete operations into one call (GET calls don't use batching) – big cost/efficiency win vs. one call per operation.
- **V2 vs V4:** V2 defaults to XML (JSON still available); query options root-only; MERGE for partial updates. V4 defaults to JSON only; query options work on expanded entities too; native PATCH; generally more performant.